Algoritmos y Estructuras de Datos

Guía Práctica 4
Verificación de programas (Parte 1)
Segundo Cuatrimestre 2026

Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

4.1. Precondición más débil en SmallLang

Ejercicio 1. Calcular las siguientes expresiones, donde a, b son variables reales, i una variable entera y A es una secuencia de reales.

a) $\text{def}(a + 1)$
b) $\text{def}(a/b)$
c) $\text{def}(\sqrt{a/b})$
d) $\text{def}(A[i] + 1)$
e) $\text{def}(A[i + 2])$
f) $\text{def}(0 \leq i \leq |A| \land A[i] \geq 0)$
g) $\text{def}(0 \leq i \leq |A| \land_L A[i] \geq 0)$

Solución

Aclaración: Recordar que siempre las variables y los valores (enteros, booleanos, etc.) se encuentran definidos.

a) $\text{def}(a + 1) \equiv \text{def}(a) \land \text{def}(1) \equiv True$

b) $\text{def}(a/b) \equiv \text{def}(a) \land (\text{def}(b) \land_L b \neq 0) \equiv b \neq 0$

c) $\text{def}(\sqrt{a/b}) \equiv \text{def}(a/b) \land_L (a/b) \geq 0 \equiv b \neq 0 \land a/b \geq 0$

d) $\text{def}(A[i] + 1) \equiv (\text{def}(A) \land \text{def}(i)) \land_L 0 \leq i < |A| \equiv 0 \leq i < |A|$

e) $\text{def}(A[i + 2]) \equiv (\text{def}(A) \land \text{def}(i)) \land_L 0 \leq i + 2 < |A| \equiv 0 \leq i + 2 < |A|$

f) $\text{def}(0 \leq i \leq |A| \ \land \ A[i] \geq 0) \equiv \text{def}(0 \leq i \leq |A|) \land \text{def}(A[i] \geq 0) \equiv True \land 0 \leq i < |A| \equiv 0 \leq i < |A|$

g) $\text{def}(0 \leq i \leq |A| \ \land_L \ A[i] \geq 0) \equiv \text{def}(0 \leq i \leq |A|) \land (0 \leq i \leq |A| \rightarrow \text{def}(A[i] \geq 0))$
$\equiv True \land (0 \leq i \leq |A| \rightarrow 0 \leq i < |A|) \equiv i \neq |A|$

Nota sobre la diferencia entre $\land$ y $\land_L$:

- El **ítem f** utiliza la conjunción estricta ($\land$). Por su definición formal ($def(P \land Q) \equiv def(P) \land def(Q)$), ambos lados se evalúan obligatoriamente. Si $i = -1$, el primer lado es falso, pero el lenguaje intenta evaluar $A[-1]$ de todas formas, por lo que la expresión se indefine. Por eso, el dominio de definición exige estrictamente que $0 \leq i < |A|$.
- El **ítem g** utiliza el operador de cortocircuito ($\land_L$). Su definición ($def(P \land_L Q) \equiv def(P) \land (P \rightarrow def(Q))$) protege de las indefiniciones: el lado derecho solo se evalúa sólo si el izquierdo es verdadero. El único caso que hace falsa a la implicación lógica ($0 \leq i \leq |A| \rightarrow 0 \leq i < |A|$) es cuando el antecedente es verdadero pero el consecuente falso, lo cual ocurre **únicamente** si $i = |A|$. Si i toma un valor fuera de rango como $i = -1$, el antecedente es falso, la implicación se vuelve verdadera, el operador frena la evaluación y la expresión global no se indefine.

Ejercicio 2. Calcular las siguientes precondiciones más débiles, donde a, b son variables reales, i una variable entera y A es una secuencia de reales.

a) $wp(\textbf{a := a+1; b := a/2}, b \geq 0)$

b) $wp(\textbf{a := A[i] + 1; b := a*a}, b \neq 2)$

c) $wp(\textbf{a := A[i] + 1; a := b*b}, a \geq 0)$

d) $wp(\textbf{a := a-b; b := a+b}, a \geq 0 \land b \geq 0)$

Solución

a)

$$\begin{aligned} wp(\textbf{a := a+1; b := a/2}, b \geq 0) &\equiv wp(\textbf{a := a+1}, wp(\textbf{b := a/2}, b \geq 0)) \\ wp(\textbf{b := a/2;}, b \geq 0) &\equiv \text{def}(a) \wedge_L a/2 \geq 0 \\ &\equiv a/2 \geq 0 \\ wp(\textbf{a := a+1}, a/2 \geq 0) &\equiv \text{def}(a) \wedge_L (a+1)/2 \geq 0 \\ &\equiv \boxed{(a+1)/2 \geq 0} \end{aligned}$$

b)

$$\begin{aligned} wp(\textbf{a := A[i] + 1; b := a*a}, b \neq 2) &\equiv wp(\textbf{a := A[i] + 1}, wp(\textbf{b := a*a}, b \neq 2)) \\ wp(\textbf{b := a*a}, b \neq 2) &\equiv \text{def}(a) \wedge_L a * a \neq 2 \\ &\equiv a * a \neq 2 \\ wp(\textbf{a := A[i] + 1}, a * a \neq 2) &\equiv (\text{def}(A) \wedge 0 \leq i < |A|) \wedge_L (A[i]+1) \times (A[i]+1) \neq 2 \\ &\equiv 0 \leq i < |A| \wedge_L A[i]^2 + 2 \times A[i] + 1 \neq 2 \\ &\equiv 0 \leq i < |A| \wedge_L A[i]^2 + 2 \times A[i] - 1 \neq 0 \\ &\equiv \boxed{0 \leq i < |A| \wedge_L A[i] \neq -1 \pm \sqrt{2}} \end{aligned}$$

c)

$$\begin{aligned} wp(\textbf{a := A[i] + 1; a := b*b}, a \geq 0) &\equiv wp(\textbf{a := A[i] + 1}, wp(\textbf{a := b*b}, a \geq 0)) \\ wp(\textbf{a := b*b}, a \geq 0) &\equiv \text{def}(b) \wedge_L b * b \geq 0 \\ &\equiv True \\ wp(\textbf{a := A[i] + 1}, True) &\equiv (\text{def}(A) \wedge 0 \leq i < |A|) \wedge_L True \\ &\equiv \boxed{0 \leq i < |A|} \end{aligned}$$

d)

$$\begin{aligned} wp(\textbf{a := a-b; b := a+b}, a \geq 0 \wedge b \geq 0) &\equiv wp(\textbf{a := a-b}, wp(\textbf{b := a+b}, a \geq 0 \wedge b \geq 0)) \\ wp(\textbf{b := a+b}, a \geq 0 \wedge b \geq 0) &\equiv (\text{def}(b) \wedge \text{def}(a)) \wedge_L (a \geq 0 \wedge a + b \geq 0) \\ &\equiv a \geq 0 \wedge a + b \geq 0 \\ wp(\textbf{a := a-b}, a \geq 0 \wedge a + b \geq 0) &\equiv ((\text{def}(b) \wedge \text{def}(a)) \wedge_L (a - b) \geq 0 \wedge (a - b) + b \geq 0 \\ &\equiv \boxed{a \geq b \wedge a \geq 0} \end{aligned}$$

Ejercicio 3. Sea $Q \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |A| \rightarrow_L A[j] \geq 0)\}$. Calcular las siguientes precondiciones más débiles, donde i es una variable entera y A es una secuencia de enteros.

a) $wp(\textbf{A[i] := 0}, Q)$

b) $wp(\textbf{A[i+2] := 0}, Q)$

c) $wp(\textbf{A[i+2] := -1}, Q)$

d) $wp(\textbf{A[i] := 2 * A[i]}, Q)$

e) $wp(\textbf{A[i] := A[i-1]}, Q)$

Solución

a)

$$
\begin{aligned}
wp(\mathbf{A[i]\ :=\ 0}, Q) &\equiv wp(setAt(A,i,0), Q) \\
&\equiv ((\text{def}(A) \wedge \text{def}(i)) \wedge_L 0 \leq i < |A|) \wedge_L Q^A_{setAt(A,i,0)} \\
&\equiv 0 \leq i < |A| \wedge_L Q^A_{setAt(A,i,0)} \\
&\equiv 0 \leq i < |A| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |setAt(A,i,0)| \rightarrow_L setAt(A,i,0)[j] \geq 0) \quad (1) \\
&\equiv 0 \leq i < |A| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |A| \rightarrow_L ((i = j \rightarrow 0 \geq 0) \wedge (i \neq j \rightarrow A[j] \geq 0))) \quad (2) \\
&\equiv 0 \leq i < |A| \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |A| \wedge i \neq j) \rightarrow_L A[j] \geq 0)
\end{aligned}
$$

Como la operación *setAt* no modifica la longitud de la secuencia, se tiene que $|setAt(A,i,0)| = |A|$ en (1).

En (2) se usa la siguiente definición,

$$
setAt(A,i,0)[j] = \begin{cases} 0 & \text{si } i = j \\ A[j] & \text{si } i \neq j \end{cases}
$$

y como $0 \geq 0 \equiv True$ se puede ver que $i = j \rightarrow 0 \geq 0$ es una tautología, por lo que el valor de verdad del consecuente del cuantificador universal sólo depende del segundo término de la conjunción, es decir, de $i \neq j \rightarrow A[j] \geq 0$.

b)

$$
\begin{aligned}
wp(\mathbf{A[i+2]\ :=\ 0}, Q) &\equiv wp(setAt(A,i+2,0), Q) \\
&\equiv ((\text{def}(A) \wedge \text{def}(i)) \wedge_L (0 \leq i+2 < |A|)) \wedge_L Q^A_{setAt(A,i+2,0)} \\
&\equiv (0 \leq i+2 < |A|) \wedge_L Q^A_{setAt(A,i,0)} \\
&\equiv (0 \leq i+2 < |A|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |setAt(A,i+2,0)| \rightarrow_L setAt(A,i+2,0)[j] \geq 0) \\
&\equiv (0 \leq i+2 < |A|) \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |A| \wedge i+2 \neq j) \rightarrow_L A[j] \geq 0)
\end{aligned}
$$

c)

$$
\begin{aligned}
wp(\mathbf{A[i+2]\ :=\ -1}, Q) &\equiv wp(setAt(A,i+2,-1), Q) \\
&\equiv ((\text{def}(A) \wedge \text{def}(i)) \wedge_L (0 \leq i+2 < |A|)) \wedge_L Q^A_{setAt(A,i+2,-1)} \\
&\equiv (0 \leq i+2 < |A|) \wedge_L Q^A_{setAt(A,i,-1)} \quad (1) \\
&\equiv (0 \leq i+2 < |A|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |setAt(A,i+2,-1)| \rightarrow_L setAt(A,i+2,-1)[j] \geq 0) \quad (2) \\
&\equiv (0 \leq i+2 < |A|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |A| \rightarrow_L ((i+2 = j \rightarrow -1 \geq 0) \wedge (i+2 \neq j \rightarrow A[j] \geq 0))) \\
&\equiv (0 \leq i+2 < |A|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |A| \rightarrow_L (False \wedge (i+2 \neq j \rightarrow A[j] \geq 0))) \\
&\equiv (0 \leq i+2 < |A|) \wedge_L |A| = 0) \\
&\equiv (0 \leq i+2 < |A|) \wedge_L |A| = 0) \\
&\equiv False
\end{aligned}
$$

En (1) se puede ver que cuando $i + 2 = j$ no se va a poder cumplir el consecuente. Esto sucede, intuitivamente, porque Q nos pide que todos los valores de la secuencia sean cero o positivos, pero estamos fijando un valor en -1. Claramente no vamos a poder cumplir con Q a menos que la secuencia esté vacía.

Pero en (2) se nos requiere que $0 \leq i + 2 < |A|$, es decir, $0 < |A|$ y $0 = |A|$. Esto resulta en una contradicción.

d)

$$
\begin{aligned}
wp(\mathbf{A[i]\ :=\ 2\ *\ A[i]}, Q) &\equiv wp(setAt(A,i,2*A[i]), Q) \\
&\equiv ((\text{def}(A) \wedge \text{def}(i)) \wedge_L (0 \leq i < |A|)) \wedge_L Q^A_{setAt(A,i,2*A[i])} \\
&\equiv (0 \leq i < |A|) \wedge_L Q^A_{setAt(A,i,2*A[i])} \\
&\equiv (0 \leq i < |A|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |setAt(A,i,2*A[i])| \rightarrow_L setAt(A,i,2*A[i])[j] \geq 0) \\
&\equiv (0 \leq i < |A|) \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |A|) \rightarrow_L setAt(A,i,2*A[i])[j] \geq 0) \\
&\equiv (0 \leq i < |A|) \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |A|) \rightarrow_L A[j] \geq 0)
\end{aligned}
$$

Intuitivamente, como en cada posición duplico el valor que tenía previamente, es claro ver que necesito tener un valor que previamente hubiera sido cero o positivo, pues $2 * k \geq 0 \leftrightarrow k \geq 0/2 \leftrightarrow k \geq 0$.

e)

$$
\begin{aligned}
wp(\textbf{A[i] := A[i-1]}, Q) &\equiv wp(setAt(A, i, A[i-1]), Q) \\
&\equiv ((\text{def}(A) \wedge \text{def}(i) \wedge \text{def}(A[i-1])) \wedge_L (0 \leq i < |A|)) \wedge_L Q^A_{setAt(A,i,A[i-1])} \\
&\equiv ((\text{def}(A) \wedge \text{def}(i) \wedge (\text{def}(A) \wedge_L 0 \leq i-1 < |A|)) \wedge_L (0 \leq i < |A|)) \wedge_L Q^A_{setAt(A,i,A[i-1])} \\
&\equiv (1 \leq i < |A|) \wedge_L Q^A_{setAt(A,i,A[i-1])} \\
&\equiv (1 \leq i < |A|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |setAt(A, i, A[i-1])| \rightarrow_L setAt(A, i, A[i-1])[j] \geq 0) \\
&\equiv (1 \leq i < |A|) \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |A| \wedge j \neq i) \rightarrow_L A[j] \geq 0)
\end{aligned}
$$

Ejercicio 4. Para los siguientes pares de programas S y postcondiciones Q

- Elegir una precondición P lo más débil posible que se corresponda con S y Q.
- Escribir P como una fórmula de lógica de primer orden.
- Mostrar formalmente que $P = wp(S, Q)$, o en caso de que no lo sea, explicar el significado de las diferencias.

a) $S \equiv$

```
if( a < 0 )
  b := a
else
  b := -a
endif
```

$Q \equiv (b = -|a|)$

b) $S \equiv$

```
if( a < 0 )
  b := a
else
  b := -a
endif
```

$Q \equiv (b = |a|)$

e) $S \equiv$

```
if( s[i] < 0 )
  s[i] := -s[i]
else
  skip
endif
```

$Q \equiv 0 \leq i < |s| \ \wedge_L \ s[i] \geq 0$

c) $S \equiv$

```
if( i > 0 )
  s[i] := 0
else
  s[0] := 0
endif
```

$Q \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \geq 0)$

d) $S \equiv$

```
if( i > 1 )
  s[i] := s[i-1]
else
  s[i] := 0
endif
```

$Q \equiv (\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L s[j] = s[j-1])$

f) $S \equiv$

```
if( s[i] > 0 )
  s[i] := -s[i]
else
  skip
endif
```

$Q \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \ \rightarrow_L \ s[j] \geq 0)$

Solución

Sabemos que $wp(\mathsf{IfThenElse}(B, S1, S2), Q) \equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q)))$

a) $\mathbf{B} \equiv a < 0$, $\mathbf{S1} \equiv b := a$, $\mathbf{S2} \equiv b := -a$, $\mathbf{Q} \equiv (b = -|a|)$

$$\begin{aligned}
wp(\mathsf{IfThenElse}(B, S1, S2), Q) &\equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q))) \\
&\equiv \text{def}(a) \wedge_L (\\
&\quad (a < 0 \wedge (\text{def}(a) \wedge_L a = -|a|)) \vee \\
&\quad (a \geq 0 \wedge (\text{def}(a) \wedge_L -a = -|a|)) \\
&) \\
&\equiv ((a < 0 \wedge a = -|a|) \vee (a \geq 0 \wedge a = |a|)) \\
&\equiv True
\end{aligned}$$

b) $\mathbf{B} \equiv a < 0$, $\mathbf{S1} \equiv b := a$, $\mathbf{S2} \equiv b := -a$, $\mathbf{Q} \equiv (b = |a|)$

$$\begin{aligned}
wp(\mathsf{IfThenElse}(B, S1, S2), Q) &\equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q))) \\
&\equiv \text{def}(a) \wedge_L (\\
&\quad (a < 0 \wedge (\text{def}(a) \wedge_L a = |a|)) \vee \\
&\quad (a \geq 0 \wedge (\text{def}(a) \wedge_L -a = |a|)) \\
&) \\
&\equiv ((a < 0 \wedge a = |a|) \vee (a \geq 0 \wedge -a = |a|)) \\
&\equiv (False \vee (a \geq 0 \wedge -a = |a|)) \\
&\equiv (a \geq 0 \wedge -a = |a|) \\
&\equiv (a = 0 \wedge -a = |a|) \vee (a > 0 \wedge -a = |a|) \\
&\equiv (a = 0 \wedge -a = |a|) \vee False \\
&\equiv (a = 0 \wedge -a = |a|) \\
&\equiv (a = 0)
\end{aligned}$$

c) $\mathbf{B} \equiv i > 0$, $\mathbf{S1} \equiv s[i] := 0$, $\mathbf{S2} \equiv s[0] := 0$, $\mathbf{Q} \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \geq 0)$

$$\begin{aligned}
wp(\mathsf{IfThenElse}(B, S1, S2), Q) &\equiv \text{def}(B) \wedge_L \big((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q))\big) \\
&\equiv \text{def}(i) \wedge_L (\\
&\quad (i > 0 \wedge (((\text{def}(s) \wedge \text{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L Q^s_{setAt(s,i,0)})) \vee \\
&\quad (i \leq 0 \wedge (((\text{def}(s) \wedge \text{def}(i)) \wedge_L 0 \leq 0 < |s|) \wedge_L Q^s_{setAt(s,0,0)})) \\
&) \\
&\equiv (0 < i < |s| \wedge_L Q^s_{setAt(s,i,0)}) \vee (i \leq 0 \wedge 0 < |s| \wedge_L Q^s_{setAt(s,0,0)}) \\
&\equiv (0 < i < |s| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] \geq 0)) \vee \\
&\quad (i \leq 0 \wedge 0 < |s| \wedge_L (\forall j : \mathbb{Z})\ (0 < j < |s| \rightarrow_L s[j] \geq 0))
\end{aligned}$$

d) $\mathbf{B} \equiv i > 1$, $\mathbf{S1} \equiv s[i] := s[i-1]$, $\mathbf{S2} \equiv s[i] := 0$, $\mathbf{Q} \equiv (\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L s[j] = s[j-1])$

$wp(\mathsf{IfThenElse}(B, S1, S2), Q) \equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q)))$

$\equiv \text{def}(i) \wedge_L ($

$(i > 1 \wedge (((\text{def}(s) \wedge \text{def}(i) \wedge 0 \leq i - 1 < |s|) \wedge_L 0 \leq i < |s|) \wedge_L Q^s_{setAt(s,i,s[i-1])})) \vee$

$(i \leq 1 \wedge (((\text{def}(s) \wedge \text{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L Q^s_{setAt(s,i,0)}))$

$)$

$\equiv ((1 < i < |s| \wedge_L Q^s_{setAt(s,i,s[i-1])}) \vee (i < |s| \wedge 0 \leq i \leq 1 \wedge_L Q^s_{setAt(s,i,0)}))$

$\equiv ($

$(1 < i < |s| \wedge_L$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L setAt(s, i, s[i-1])[j] = setAt(s, i, s[i-1])[j-1])) \vee$

$(i < |s| \wedge 0 \leq i \leq 1 \wedge_L$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L setAt(s, i, 0)[j] = setAt(s, i, 0)[j-1]))$

$)$

$\equiv ($

$(1 < i < |s| \wedge_L ((\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge i \neq j \wedge i \neq j - 1 \rightarrow_L s[j] = s[j-1])) \wedge$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge i = j \rightarrow_L setAt(s, i, s[i-1])[j] = setAt(s, i, s[i-1])[j-1])) \wedge$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge i = j - 1 \rightarrow_L$

$setAt(s, i, s[i-1])[j] = setAt(s, i, s[i-1])[j-1])))$

$\vee$

$(i < |s| \wedge_L (i = 0 \wedge_L$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L setAt(s, i, 0)[j] = setAt(s, i, 0)[j-1])) \vee$

$(i = 1 \wedge_L$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L setAt(s, i, 0)[j] = setAt(s, i, 0)[j-1]))$

$))$

$\equiv ($

$(1 < i < |s| \wedge_L ((\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge j \neq i \wedge j \neq i + 1 \rightarrow_L s[j] = s[j-1])) \wedge$

$true \wedge (0 \leq i + 1 < |s| \rightarrow_L s[i-1] = s[i+1]))$

$\vee$

$(i < |s| \wedge_L (i = 0 \wedge_L (\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge j - 1 \neq i \rightarrow_L s[j] = s[j-1])$

$\wedge (1 < |s| \rightarrow_L s[1] = 0) \vee$

$(i = 1 \wedge_L (\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge j \neq i \wedge j - 1 \neq i \rightarrow_L s[j] = s[j-1])$

$\wedge (0 < |s| \rightarrow_L s[0] = 0) \wedge (2 < |s| \rightarrow_L s[2] = 0)))$

$))$

En el lado izquierdo del $\vee$, veo que el rango de i en el antecedente de la implicancia va desde $-1 \leq i < |s| - 1$, pero estoy en el caso $i > 1$, por lo que al juntarlos su rango es $1 < i < |s| - 1$, como ya tengo un $1 < i < |s|$ al principio, *sé* que ya necesariamente i va a ser > 1, entonces puedo dejar solo el otro extremo del rango

Respecto al lado derecho del $\vee$ noto que para que se cumpla todo eso,

los otros elementos de la secuencia deben ser igual a 0

$\equiv (($

$1 < i < |s| \wedge_L$

$(\forall j : \mathbb{Z})\ (1 \leq j < |s| \wedge j \neq i \wedge j \neq i + 1 \rightarrow_L s[j] = s[j-1]) \wedge$

$(i < |s| - 1 \rightarrow_L s[i+1] = s[i-1])$

$) \vee (i < |s| \wedge 0 \leq i \leq 1 \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] = 0))$

$)$

e) $\mathbf{B} \equiv s[i] < 0$, $\mathbf{S1} \equiv s[i] := -s[i]$, $\mathbf{S2} \equiv skip$, $\mathbf{Q} \equiv (0 \leq i < |s| \ \wedge_L \ s[i] \geq 0)$

$$
\begin{aligned}
wp(\mathsf{IfThenElse}(B, S1, S2), Q) &\equiv \mathrm{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q))) \\
&\equiv \mathrm{def}(i) \wedge_L (\\
&\quad \big(((\mathrm{def}(s) \wedge \mathrm{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L s[i] < 0 \wedge Q^s_{setAt(s,i,-s[i])}\big) \vee \\
&\quad \big(((\mathrm{def}(s) \wedge \mathrm{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L s[i] \geq 0 \wedge Q\big) \\
&) \\
&\equiv 0 \leq i < |s| \wedge_L ((s[i] < 0 \wedge_L Q^s_{setAt(s,i,-s[i])}) \vee (s[i] \geq 0 \wedge_L Q)) \\
&\equiv 0 \leq i < |s| \wedge_L ((s[i] < 0 \wedge_L (0 \leq i < |s| \wedge setAt(s, i, -s[i])[i] \geq 0)) \vee \\
&\quad (s[i] \geq 0 \wedge_L (0 \leq i < |s| \wedge s[i] \geq 0))) \\
&\equiv 0 \leq i < |s| \wedge_L ((s[i] < 0 \wedge_L -s[i] \geq 0) \vee (s[i] \geq 0 \wedge_L s[i] \geq 0)) \\
&\equiv 0 \leq i < |s| \wedge_L ((s[i] < 0 \wedge_L s[i] \leq 0) \vee s[i] \geq 0) \\
&\equiv 0 \leq i < |s| \wedge_L (True \vee s[i] \geq 0) \\
&\equiv 0 \leq i < |s| \wedge_L True \\
&\equiv 0 \leq i < |s|
\end{aligned}
$$

f) $\mathbf{B} \equiv s[i] > 0$, $\mathbf{S1} \equiv s[i] := -s[i]$, $\mathbf{S2} \equiv skip$, $\mathbf{Q} \equiv (\forall j : \mathbb{Z}) \ (0 \leq j < |s| \ \rightarrow_L \ s[j] \geq 0)$

$$
\begin{aligned}
wp(\mathsf{IfThenElse}(B, S1, S2), Q) &\equiv \mathrm{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q))) \\
&\equiv \mathrm{def}(i) \wedge_L (\\
&\quad (((\mathrm{def}(s) \wedge \mathrm{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L s[i] > 0 \wedge Q^s_{setAt(s,i,-s[i])}) \vee \\
&\quad (((\mathrm{def}(s) \wedge \mathrm{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L s[i] \leq 0 \wedge Q) \\
&) \\
&\equiv 0 \leq i < |s| \wedge_L ((s[i] > 0 \wedge_L Q^s_{setAt(s,i,-s[i])}) \vee (s[i] \leq 0 \wedge_L Q)) \\
&\equiv s[i] > 0 \wedge (\forall j : \mathbb{Z}) \ (0 \leq j < |s| \rightarrow_L (i = j \rightarrow -s[i] > 0) \wedge (i \neq j \rightarrow s[j] > 0)) \vee \\
&\quad (s[i] \leq 0 \wedge (\forall j : \mathbb{Z}) \ (0 \leq j < |s| \rightarrow_L s[j] > 0)) \\
&\equiv (s[i] > 0 \wedge False) \vee (s[i] \leq 0 \wedge (\forall j : \mathbb{Z}) \ (0 \leq j < |s| \rightarrow_L s[j] > 0)) \\
&\equiv s[i] \leq 0 \wedge (\forall j : \mathbb{Z}) \ (0 \leq j < |s| \rightarrow_L s[j] > 0)
\end{aligned}
$$

Ejercicio 5. Para las siguientes especificaciones:

- Poner nombre al problema que resuelven
- Escribir un programa S sencillo en SmallLang, sin ciclos, que lo resuelva
- Dar la precondición más débil del programa escrito con respecto a la postcondición de su especificación

a) `proc problema1`(in $s : \mathsf{seq}\langle\mathbb{Z}\rangle$, in $i : \mathbb{Z}$, inout $a : \mathbb{Z}$) {
   `requiere` { $0 \leq i < |s| \ \wedge_L \ a = \sum_{j=0}^{i-1} s[j]$ }
   `asegura` { $a = \sum_{j=0}^{i} s[j]$ }
}

b) `proc problema2`(in $s : \mathsf{seq}\langle\mathbb{Z}\rangle$, in $i : \mathbb{Z}$) : bool {
   `requiere` { $0 \leq i < |s| \ \wedge_L \ (\forall j : \mathbb{Z}) \ (0 \leq j < i \rightarrow_L s[j] \geq 0)$ }
   `asegura` { $res = true \leftrightarrow (\forall j : \mathbb{Z}) \ (0 \leq j \leq i \rightarrow_L s[j] \geq 0)$ }
}

c) `proc problema3`(inout $s : \mathsf{seq}\langle\mathbb{Z}\rangle$, in $i : \mathbb{Z}$) {
   `requiere` { $(0 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z}) \ (0 \leq j < i \rightarrow s[j] = fibonacci(j))$ }
   `asegura` { $(\forall j : \mathbb{Z}) \ (0 \leq j \leq i \rightarrow s[j] = fibonacci(j))$ }

}

Solución

a)
```
agregaI-EsimoALaSuma
a := a + s[i];
return a;
```

b)
```
hastaI-EsimoSonPositivos
return s[i] >= 0;
```

c)
```
sucesionDeFibonacciHastaI
if i >= 2
      s[i] := s[i-1] + s[i-2];
else
      // Como s[0] = 0 y s[1] = 1...
      s[i] := i;

return s;
```

Ejercicio 6. Dada la poscondición $Q \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \mod 2 = 0)\}$ y el siguiente código

```
if (i mod 3 = 0)
  s[i] := s[i] + 6;
else
  s[i] := i;
endif
```

a) Demostrar que las siguientes fórmulas no son $wp(S, Q)$, mediante contraejemplos. Puede demostrar que no se satisface la tripla de Hoare $\{P\}$ **S** $\{Q\}$, o bien demostrar que P como precondición no es la más débil.
Ayuda:
Un ejemplo que muestra que no se cumple $\{P\}$ ***S*** $\{Q\}$ *es un ejemplo en el que se satisface* P*, pero luego de ejecutar* S *no se satisface* Q*.*
Un ejemplo que muestra que no P *no es la precondición más débil es un ejemplo en el que no se satisface* P*, pero luego de ejecutar* S *sí se satisface* Q*.*

i) $P \equiv \{0 \leq i \leq |s| \wedge_L i \ mod\ 3 = 0 \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j]\ mod\ 2 = 0)\}$
ii) $P \equiv \{0 \leq i < |s| \wedge_L i \ mod\ 3 \neq 0 \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j]\ mod\ 2 = 0)\}$
iii) $P \equiv \{i \ mod\ 3 = 0 \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j]\ mod\ 2 = 0)\}$
iv) $P \equiv \{0 \leq i < |s|/2 \wedge_L i \ mod\ 3 = 0 \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j]\ mod\ 2 = 0)\}$

Solución

a) i) Si tomo un i que no sea múltiplo de 3 pero que sea par, al entrar a la rama *else* se le asignará el valor de la posición al elemento de la secuencia. De esta manera, la posición seguirá teniendo un valor par. Por lo tanto, encontramos un valor que hace verdadera la poscondición pero no está contemplada en la WP presentada.
Contraejemplo. $i = 2, s = [0, 4, 20, 10, 8, 6]$

ii) Si tomo un i que sea múltiplo de 3, al entrar a la rama *then* se le sumará seis al valor de esa posición. De esta manera, la posición seguirá teniendo un valor par ya que por precondición tenía un valor par y el seis es un número par. Por lo tanto, encontramos un valor que hace verdadera la poscondición pero no está contemplada en la WP presentada.
Contraejemplo. $i = 3, s = [0, 4, 20, 10, 8, 6]$

iii) La precondición presentada permite recibir valores de i que no estén en rango. Esto provocará un error al intentar escribir en esa posición de la secuencia.
Contraejemplo. $i = -3, s = [0, 4, 20, 10, 8, 6]$

iv) Si tomo un i que se encuentre entre $|s|/2$ y $|s|$, al entrar en cualquiera de las dos ramas se obtendrá un valor par en esa posición según lo explicado en los ítems I y II. Por lo tanto, encontramos un valor que hace verdadera la poscondición pero no está contemplada en la WP presentada.
Contraejemplo. $i = 4, s = [0, 4, 20, 10, 8, 6]$

4.1.1. Ejercicios de parcial

Ejercicio 7. Dado el siguiente condicional determinar la precondición más débil que permite hacer valer la poscondición (Q) propuesta. Se pide:

- Describir en palabras la WP esperada
- Derivarla formalmente a partir de los axiomas de precondición más débil. Para obtener el puntaje máximo deberá simplificarla lo más posible.

a) $Q \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] > 0)\}$

```
if (s[i] < 0)
  s[i] := - s[i]
else
  s[i] := 0
endif
```

b) $Q \equiv \{(\exists j : \mathbb{Z})\ (j \geq 0 \wedge j^2 = a)\}$

```
if (a mod 2 = 0)
  a := a * a
else
  a := - |a|
endif
```

c) $Q \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] = 2^j)\}$

```
if (s[i] != 2^i)
  s[i] = 2 * s[i-1]
else
  s[0] = 1
endif
```

d) $Q \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j]\ mod\ 3 = 0)\}$

```
if (i mod 3 == 0)
  s[i] = s[i] + 6
else
  s[i] = i
endif
```

e) $Q \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j]\ mod\ 2 = 0)\}$

```
if (i mod 2 = 0)
  s[i] = 2 * s[i]
else
  s[0] = 3
endif
```

Solución

a) La precondición más débil debería requerir que i esté en rango para que no se indefina el código, que el valor en la posición i-ésima sea negativo y que las demás posiciones del arreglo tengan valores positivo.

Se puede ver que si el valor de la posición i-ésima no fuera negativo, entonces el código S pondría un cero en esa posición y no se cumpliría la poscondición Q.

Ahora vamos a calcularla **formalmente**.

B $\equiv s[i] < 0$, **S1** $\equiv s[i] := -s[i]$, **S2** $\equiv s[i] := 0$, **Q** $\equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \ \rightarrow_L \ s[j] > 0)$

$$
\begin{aligned}
wp(\mathsf{IfThenElse}(B, S1, S2), Q) &\equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q))) \\
&\equiv ((\text{def}(s) \wedge \text{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L (\\
&\quad (((\text{def}(s) \wedge \text{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L s[i] < 0 \wedge Q^s_{setAt(s,i,-s[i])}) \vee \\
&\quad (((\text{def}(s) \wedge \text{def}(i)) \wedge_L 0 \leq i < |s|) \wedge_L s[i] \geq 0 \wedge Q^s_{setAt(s,i,0)}) \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (\\
&\quad (s[i] < 0 \wedge_L Q^s_{setAt(s,i,-s[i])}) \vee \\
&\quad (s[i] \geq 0 \wedge_L Q^s_{setAt(s,i,0)}) \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (\\
&\quad (s[i] < 0 \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, i, -s[i])[j] > 0)) \vee \\
&\quad (s[i] \geq 0 \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, i, 0)[j] > 0)) \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (\\
&\quad (s[i] < 0 \wedge_L ((\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0) \wedge -s[i] > 0)) \vee \\
&\quad (s[i] \geq 0 \wedge_L ((\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0) \wedge 0 > 0)) \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (\\
&\quad (s[i] < 0 \wedge_L ((\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0) \wedge True)) \vee \\
&\quad (s[i] \geq 0 \wedge_L ((\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0) \wedge False)) \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (\\
&\quad (s[i] < 0 \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0)) \vee \\
&\quad (s[i] \geq 0 \wedge_L False) \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (\\
&\quad (s[i] < 0 \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0)) \vee \\
&\quad False \\
&) \\
&\equiv (0 \leq i < |s|) \wedge_L (s[i] < 0 \wedge_L (\forall j : \mathbb{Z})\ ((0 \leq j < |s| \wedge i \neq j) \rightarrow_L s[j] > 0))
\end{aligned}
$$

Aclaración. Se puede ver que $-s[i] > 0$ es una tautología (evalúa siempre a $True$) porque justamente B nos está diciendo que vale que $s[i] < 0$. En el otro caso, es evidente por qué $0 > 0$ es una contradicción, lo que termina cancelando todo el término correspondiente a la rama *else* del condicional.

b) se sabe que al finalizar el programa, a tiene que ser un *cuadrado perfecto*. Si se cumple la guarda (o sea, si a es par), entonces a pasa a ser $a * a$, o sea, un cuadrado perfecto. Por lo tanto se va a cumplir la postcondición no importa cuál sea el valor de a. En cambio si no se cumple la guarda (si a es impar), a se va a transformar en un número negativo. Como un número negativo nunca puede ser un cuadrado perfecto, nunca se va a poder cumplir la postcondición. En resumen, para que se cumpla la postcondición, *a tiene que ser par.*

Veamos formalmente: **B** $\equiv a \ \text{ mód } 2 = 0$, **S1** $\equiv a := a * a$, **S2** $\equiv a := -|a|$, **Q** $\equiv (\exists j : \mathbb{Z})\ (j \geq 0 \wedge j^2 = a)$

$wp(\mathsf{IfThenElse}(B, S1, S2), Q) \equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q)))$

$wp(S1, Q) \equiv def(a * a) \wedge_L Q^a_{a*a} \equiv (\exists j : \mathbb{Z})\ (j \geq 0 \wedge j^2 = a * a)$
esto es *verdadero* pues para $j = |a|$ se cumple que $j^2 = a * a$

$wp(S2, Q) \equiv def(-|a|) \wedge_L Q^a_{-|a|} \equiv (\exists j : \mathbb{Z})\ (j \geq 0 \wedge j^2 = -|a|)$
esto es *falso* pues no existe ningún número que elevado al cuadrado de negativo.

Uniendo todo:

$wp(\mathsf{IfThenElse}(B, S1, S2), Q) \equiv ((a \mod 2 = 0) \wedge true) \vee ((a \mod 2 = 0) \wedge false) \equiv a \mod 2 = 0$

que es lo que esperábamos.

c) La postcondición dice que la secuencia s tiene todas las potencias de dos ordenadas. En el código, dado un índice i, si la secuencia original no tiene el valor esperado en esa posición, entonces se asigna el valor esperado (nótese que $2^i = 2 * 2^{i-1}$). En cambio si en la posición i ya está el valor esperado, se le asigna 1 a la posición 0 (la potencia correcta). Luego la wp esperada sería que el valor de i está en rango y que la secuencia s tiene las potencias de dos ordenadas en todas las posiciones salvo en la i o en la 0. En esas posiciones puede haber cualquier cosa (en una pero no en ambas).

Veamos la derivación formal:

$\mathbf{B} \equiv s[i] \neq 2^j$, $\mathbf{S1} \equiv s[i] := 2 * s[i-1]$, $\mathbf{S2} \equiv s[0] := 1$, $\mathbf{Q} \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] = 2^j)$

$wp(\mathsf{IfThenElse}(B, S1, S2), Q) \equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q)))$

$wp(S1, Q) \equiv 0 \leq i < |s| \wedge_L \text{def}(2 * s[i-1]) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, i, 2 * s[i] - 1][j] = 2^j) \equiv$
$(1 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, i, 2 * s[i-1])[j] = 2^j)$

Dividimos en dos casos: $i = j$ y $i \neq j$. Vamos a hacer dos paratodos diferentes:

$(1 \leq i < |s|) \wedge_L ((\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i = j \rightarrow_L 2 * s[j-1] = 2^j) \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] = 2^j))$

Nótese en el primer paratodo que el $s[j-1]$ está incluído en el segundo paratodo, por lo cuál vale que $s[j-1] = 2^j$. Por lo tanto $2 * s[j-1] = 2^j$ se va a cumplir siempre (si se cumple el segundo paratodo). Luego, el primer paratodo es verdadero:

$(1 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] = 2^j)$

Veamos la segunda parte:

$wp(S1, Q) \equiv 0 \leq i < |s| \wedge_L \text{def}(0) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, 0, 1)[j] = 2^j)$

Dividimos en dos casos: $j = 0$ y $j \neq 0$. Vamos a hacer dos paratodos diferentes:

El primer paratodo tiene un solo caso y la condición es siempre verdadera ($1 = 2^0$). Por lo tanto nos queda:

$(0 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L s[j] = 2^j)$

Juntando todo:

$(0 \leq i < |s|) \wedge_L ((s[i] \neq 2^i) \wedge (1 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] = 2^j)) \vee$
$((s[i] = 2^i) \wedge (0 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L s[j] = 2^j)) \equiv$

$((1 \leq i < |s|) \wedge_L s[i] \neq 2^i \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] = 2^j)) \vee$
$((0 \leq i < |s|) \wedge_L s[i] = 2^i \wedge (\forall j : \mathbb{Z})\ (1 \leq j < |s| \rightarrow_L s[j] = 2^j))$

d) La postcondición dice que al salir del programa todos los elementos de la secuencia s deben ser múltiplo de 3. El programa toma un valor i y si ese valor es múltiplo de 3, le suma 6 a esa posición. Nótese que sumar 6 no cambia el hecho de que el número sea múltiplo de 3. Si i no es múltiplo de 3, le asigna i a esa posición. O sea que en la posición i vamos a tener un número que no es múltiplo de 3. Por lo tanto, si el programa pasa por esa rama, no se va a cumplir la postcondición. En conclusión, la wp debería decir que la guarda se cumple (i está en rango y es múltiplo de 3) y que todos los elementos de la secuencia son múltiplos de 3.

Veamos la derivación formal: $\mathbf{B} \equiv i \mod 3 = 0$, $\mathbf{S1} \equiv s[i] := s[i] + 6$, $\mathbf{S2} \equiv s[i] := i$, $\mathbf{Q} \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \mod 3 = 0)$

$wp(\mathsf{IfThenElse}(B, S1, S2), Q) \equiv \text{def}(B) \wedge_L ((B \wedge wp(S1, Q)) \vee (\neg B \wedge wp(S2, Q)))$

$wp(S1, Q) \equiv \text{def}(s[i] + 6) \wedge_L Q^s_{setAt(s,i,s[i]+6)} \equiv 0 \leq i < |s| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, i, s[i] + 6)[j] \mod 3 = 0)$

Dividimos en dos casos: $i = j$ y $i \neq j$. Vamos a hacer dos paratodos diferentes:

$0 \leq i < |s| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i = j \rightarrow_L s[j] + 6 \mod 3 = 0) \wedge$
$(\forall j : \mathbb{Z})\ (0 \leq j < |s| i \neq j \rightarrow_L s[j] \mod 3 = 0)$

vemos que las dos condiciones son equivalentes, por lo que las podemos unir nuevamente:

$0 \leq i < |s| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \mod 3 = 0)$

Vamos ahora con la segunda parte:

$wp(S2, Q) \equiv \text{def}(i) \wedge_L Q^s setAt(s, i, i) \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L setAt(s, i, i)[j] \mod 3 = 0)$

dividimos en dos casos: i = j y i != j. Vamos a hacer dos paratodos diferentes:

$(\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i = j \rightarrow_L i \mod 3 = 0) \wedge$
$(\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] \mod 3 = 0)$

vemos que el primer paratodo tiene un único caso, podemos escribirlo sin paratodo...

$(0 \leq j < |s| \wedge_L i \mod 3 = 0) \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] \mod 3 = 0)$

Y ahora juntamos todo...

$((i \mod 3 = 0) \wedge (0 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \mod 3 = 0)) \vee$
$((i \mod 3 \neq 0) \wedge (i \mod 3 = 0) \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge i \neq j \rightarrow_L s[j] \mod 3 = 0))$

la segunda línea es falsa ($i \mod 3 \neq 0 \wedge i \mod 3 = 0$), por lo que queda:

$(i \mod 3 = 0 \wedge (0 \leq i < |s|) \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] \mod 3 = 0))$

que era lo que esperábamos.

Ejercicio 8. Para los siguientes algoritmos **S** con sus post condiciones Q, indique cuál de las precondiciones propuestas es la *Precondición más débil* y justifique muy brevemente en palabras.

a) **S** ≡

```
if (a mod 2 = 0)
  a := |a| + 1
else
  a := |a| * 2
endif
```

$Q \equiv \{a \mod 2 = 0\}$

(i) $P \equiv \{a \mod 2 = 0\}$

(ii) $P \equiv \{a \mod 2 \neq 0\}$

(iii) $P \equiv \{True\}$

b) **S** ≡

```
j := i - 2
s[j] := 2 * i
```

$Q \equiv \{(\forall k : \mathbb{Z})\ (0 \leq k < |s| \rightarrow_L s[k] \mod 2 = 0)\}$

(i) $P \equiv \{(\forall k : \mathbb{Z})\ (0 \leq k < |s| \wedge k \neq i - 2 \rightarrow_L s[k] \mod 2 = 0)\}$

(ii) $P \equiv \{2 \leq i < |s| + 2 \wedge$
$(\forall k : \mathbb{Z})\ (0 \leq k < |s| \wedge k \neq i - 2 \rightarrow_L s[k] \mod 2 = 0)\}$

(iii) $P \equiv \{i \mod 2 = 0 \wedge 2 \leq i < |s| + 2 \wedge$
$(\forall k : \mathbb{Z})\ (0 \leq k < |s| \wedge k \neq i - 2 \rightarrow_L s[k] \mod 2 = 0)\}$

c) **S** ≡

```
if (x > y)
  y := x
else
  y := 3
endif
```

$Q \equiv \{y > 0\}$

(i) $P \equiv \{x > y\}$

(ii) $P \equiv \{x > y \vee x \leq y\}$

(iii) $P \equiv \{(x > 0 \wedge x > y) \vee (x \leq y)\}$

d) **S** ≡

```
if (i mod 2 = 0)
  s[i] := 1
else
  s[i] := 5
endif
```

$Q \equiv \{(\forall i : \mathbb{Z})\ (0 \leq i < |s| \wedge i \mod 2 = 0 \rightarrow_L s[i] = 1)\}$

(i) $P \equiv \{0 \leq i < |s| \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge j \mod 2 = 0 \wedge j \neq i \rightarrow_L s[j] = 1)\}$

(ii) $P \equiv \{0 \leq i < |s| \wedge (\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge j \neq i \rightarrow_L s[j] = 1)\}$

(iii) $P \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \wedge j \mod 2 = 0 \wedge j \neq i \rightarrow_L s[j] = 1)\}$

Solución

a) **Respuesta: (ii)** $P \equiv \{a \mod 2 \neq 0\}$
Justificación: Si a es impar, el programa ingresa por el `else` y multiplica $|a| \times 2$, garantizando un resultado par (cumple Q). Si fuera par (opción i), se le sumaría 1 y quedaría impar, fallando la postcondición.

b) **Respuesta: (ii)** $P \equiv \{2 \leq i < |s| + 2 \land (\forall k : \mathbb{Z})\ (0 \leq k < |s| \land k \neq i - 2 \rightarrow_L s[k] \mod 2 = 0)\}$
Justificación: Se necesita garantizar que el acceso $j = i - 2$ no indefina el arreglo $(0 \leq i - 2 < |s| \equiv 2 \leq i < |s| + 2)$. No hace falta pedir que i sea par (como hace la opción iii) porque el valor que se guarda es $2 \times i$, que siempre será par sin importar si i es impar.

c) **Respuesta: (iii)** $P \equiv \{(x > 0 \land x > y) \lor (x \leq y)\}$
Justificación: Si no se cumple la guarda $(x \leq y)$, se asigna $y := 3$, lo cual siempre es positivo y cumple Q. Pero si se cumple la guarda $(x > y)$, se asigna $y := x$, por lo tanto, para garantizar $y > 0$, es estrictamente necesario exigir que $x > 0$ de antemano.

d) **Respuesta: (i)** $P \equiv \{0 \leq i < |s| \land (\forall j : \mathbb{Z})\ (0 \leq j < |s| \land j \mod 2 = 0 \land j \neq i \rightarrow_L s[j] = 1)\}$
Justificación: La opción (iii) es incorrecta porque no evalúa que el índice i esté en rango (podría indefinirse). La opción (ii) pide innecesariamente que *todas* las demás posiciones (incluso las impares) valgan 1. La opción (i) es la más débil entre las propuestas válidas.

4.2. Demostración de correctitud de ciclos en SmallLang

4.2.1. Teorema del invariante: corrección de ciclos

Ejercicio 9. Consideremos el problema de sumar los elementos de un arreglo y la siguiente implementación en SmallLang, con el invariante del ciclo.

Especificación

```
proc sumar(in s : array⟨ℤ⟩) : ℤ {
   requiere { True }
   asegura { res = Σ_{j=0}^{|s|-1} s[j] }
}
```

Implementación en SmallLang

```
res := 0;
i := 0;
while (i < s.size()) do
   res := res + s[i];
   i := i + 1
endwhile
```

Invariante de Ciclo

$$I \equiv 0 \leq i \leq |s| \land_L res = \sum_{j=0}^{i-1} s[j]$$

a) Escribir la precondición y la poscondición del ciclo.

b) ¿Qué punto falla en la demostración de corrección si el primer término del invariante se reemplaza por $0 \leq i < |s|$?

c) ¿Qué punto falla en la demostración de corrección si el límite superior de la sumatoria $(i - 1)$ se reemplaza por i?

d) ¿Qué punto falla en la demostración de corrección si se invierte el orden de las dos instrucciones del cuerpo del ciclo?

e) Mostrar la corrección parcial del ciclo, usando los primeros puntos del teorema del invariante.

f) Proponer una función variante y mostrar la terminación del ciclo, utilizando la función variante.

Solución

a) $P_c \equiv \{res = 0 \land i = 0\}$
$Q_c \equiv \{i = |s| \land res = \sum_{j=0}^{i-1} s[j]\}$

b) Falla la demostración de preservación del invariante: $\{I \land B\}$ **ciclo** $\{I\}$.

¿Por qué no falla a la salida del ciclo $((I \land \neg B) \rightarrow Q_c)$?
Es un error intuitivo común creer que el problema radica en que i nunca alcanza el valor $|s|$ necesario para Q_c. Sin

embargo, si analizamos la implicación formalmente: La guarda negada es $\neg B \equiv i \geq |s|$. Si el invariante propuesto fuera $I \equiv 0 \leq i < |s|$, la conjunción $(I \wedge \neg B)$ resultaría en $(i < |s| \wedge i \geq |s|)$, lo cual es una contradicción lógica estricta $(False)$. En lógica matemática, una implicación con un antecedente falso $(False \rightarrow Q_c)$ evalúa como *vacuamente verdadera*. Por lo tanto, el axioma de salida del ciclo se cumple de manera trivial y no es el causante de la falla. El teorema se rompe exclusivamente en el paso de preservación: al ingresar a la iteración con $i = |s| - 1$, el incremento $i := i + 1$ genera que al final del ciclo $i = |s|$, lo cual viola la restricción $i < |s|$ requerida por $wp(S, I)$.

c) Falla la demostración de $P_c \rightarrow I$ porque antes de ingresar al ciclo se pide que en res esté almacenado el valor del primer elemento, pero no se cumple siempre porque al comenzar tenemos que $res = 0$. Además, en cada iteración se va a estar esperando un elemento más en la suma resultante. Por otro lado, al ingresar a la última iteración, se intentará acceder a una posición de la secuencia que no existe y se indefiniría la suma.

d) Falla la demostración de $\{I \wedge B\}$ **ciclo** $\{I\}$.

e)
1. $P_c \rightarrow I$. (Demostrar que vale I sabiendo que vale P_c).
   - $i = 0 \rightarrow 0 \leq 0 \leq |s|$ ✓
   - $i = 0 \rightarrow res = \sum_{j=0}^{i-1} s[j] = \sum_{j=0}^{-1} s[j] = 0$ por tener rango vacío. ✓
2. $\{I \wedge B\}$ **ciclo** $\{I\}$. Para ver que la tripla de Hoare es válida, hay que probar $I \wedge B \rightarrow wp(ciclo, I)$
   -
$$\begin{aligned} wp(\textbf{ciclo}, I) &\equiv wp(\textbf{S1;S2}, I) \\ &\equiv wp(\textbf{S1}, wp(\textbf{S2}, I)) \\ &\equiv wp(\textbf{S1}, (0 \leq i+1 \leq |s| \wedge_L res = \sum_{j=0}^{i} s[j])) \\ &\equiv 0 \leq i+1 \leq |s| \wedge_L res + s[i] = \sum_{j=0}^{i} s[j] \\ &\equiv \boxed{0 \leq i+1 \leq |s| \wedge_L res = \textstyle\sum_{j=0}^{i-1} s[j]} \end{aligned}$$
   - $I \wedge B \rightarrow wp(ciclo, I)$
     - $I \wedge B \Rightarrow 0 \leq i \leq |s| \wedge i < |s|$. Si $0 \leq i$ entonces $0 \leq i + 1$. Por el otro lado, $i < |s|$ es lo mismo que decir $i + 1 < |s| + 1 \equiv i + 1 \leq |s|$. Juntando todo llegamos a $0 \leq i + 1 \leq |s|$ como dice la WP ✓
     - $res = \sum_{j=0}^{i-1} s[j]$ está idéntico en I y en la WP ✓
3. $(I \wedge \neg B) \rightarrow Q_c$
   - $I \wedge \neg B \rightarrow 0 \leq i \leq |s| \wedge i \geq |s|$. Entonces, sabemos que $i = |s|$ ✓
   - Usando eso en el otro término de I tenemos $res = \sum_{j=0}^{i-1} s[j] = \sum_{j=0}^{|s|-1} s[j]$ ✓

f) Función variante propuesta $f_v = |s| - i$

4. $\{I \wedge B \wedge v_0 = f_v\}$ **ciclo** $\{f_v < v_0\}$ válida si $(I \wedge B \wedge v_0 = f_v) \rightarrow wp(\textbf{ciclo}, f_v < v_0)$.
   -
$$\begin{aligned} wp(\textbf{ciclo}, f_v < v_0) &\equiv wp(\textbf{S1;S2}, f_v < v_0) \\ &\equiv wp(\textbf{res := res + s[i]}, wp(\textbf{i := i + 1}, |s| - i < v_0)) \\ &\equiv wp(\textbf{res := res + s[i]}, (|s| - i - 1 < v_0)) \\ &\equiv \text{def}(s[i]) \wedge_L |s| - i - 1 < v_0 \\ &\equiv 0 \leq i < |s| \wedge_L |s| - i - 1 < v_0 \\ &\equiv \boxed{|s| - i - 1 < v_0} \end{aligned}$$
   - $(I \wedge B \wedge v_0 = f_v) \rightarrow wp(\textbf{ciclo}, f_v < v_0)$: Por $v_0 = f_v$, $|s| - i - 1 < |s| - i \leftrightarrow -1 < 0$ ✓
5. $I \wedge f_v \leq 0 \rightarrow \neg B$: $I \wedge f_v \leq 0 \rightarrow 0 \leq i \leq |s| \wedge |s| - i \leq 0 \leftrightarrow$
   $0 \leq i \leq |s| \wedge |s| \leq i \leftrightarrow$
   $i = |s| \Rightarrow \neg(i < |s|) \equiv \neg B$ ✓

Entonces el ciclo termina y, por consiguiente, es correcto.

Ejercicio 10. Dadas la especificación y la implementación del problema `sumarParesHastaN`

Especificación

```
proc sumarParesHastaN(in n : ℤ) : ℤ {
   requiere { n ≥ 0 }
   asegura {
       res = ∑_{j=0}^{n-1}(IfThenElse(j  mód 2 = 0, j, 0))
   }
}
```

Implementación en SmallLang

```
res := 0;
i := 0;
while (i < n) do
  res := res + i;
  i := i + 2
endwhile
```

Invariante de ciclo

$$I \equiv 0 \leq i \leq n+1 \land i \mod 2 = 0 \land res = \sum_{j=0}^{i-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))$$

a) Escribir la precondición y la poscondición del ciclo.

b) Mostrar la corrección parcial del ciclo, usando los primeros puntos del teorema del invariante.

c) Proponer una función variante y mostrar la terminación del ciclo, utilizando la función variante.

Solución

a) $P_c \equiv \{n \geq 0 \land res = 0 \land i = 0\}$
$Q_c \equiv \{res = \sum_{j=0}^{i-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0)\}$

b) 1. $P_c \to I$.

- $i = 0 \Rightarrow 0 \leq 0 \leq n+1$ ✓
- $i = 0 \Rightarrow 0 \bmod 2 = 0$ ✓
- $res = 0 \land i = 0 \Rightarrow res = \sum_{j=0}^{0-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0)) = 0$ porque el rango es vacío. ✓

2. $\{I \land B\}$ **ciclo** $\{I\}$. $\leftrightarrow I \land B \to wp(ciclo, I)$

- $$\begin{aligned}
wp(\textbf{ciclo}, I) &\equiv wp(\textbf{S1;S2}, I) \\
&\equiv wp\big(\textbf{res := res + i}, wp(\textbf{i := i + 2}, I)\big) \\
&\equiv wp\Big(\textbf{res := res + i}, 0 \leq i+2 \leq n+1 \land i+2 \bmod 2 = 0 \land \\
&\qquad \land res = \sum_{j=0}^{i+1}\big(\mathsf{IfThenElse}(j \bmod 2 = 0, j, 0)\big)\Big) \\
&\equiv wp\Big(\textbf{res := res + i}, 0 \leq i+2 \leq n+1 \land i \bmod 2 = 0 \land \\
&\qquad \land res = \sum_{j=0}^{i+1}\big(\mathsf{IfThenElse}(j \bmod 2 = 0, j, 0)\big)\Big) \\
&\equiv 0 \leq i+2 \leq n+1 \land i \bmod 2 = 0 \land res + i = \sum_{j=0}^{i+1}\big(\mathsf{IfThenElse}(j \bmod 2 = 0, j, 0)\big) \\
&\equiv 0 \leq i+2 \leq n+1 \land i \bmod 2 = 0 \land res = \sum_{j=0}^{i+1}\big(\mathsf{IfThenElse}(j \bmod 2 = 0, j, 0)\big) - i \\
&\equiv \boxed{0 \leq i+2 \leq n+1 \land i \bmod 2 = 0 \land res = \sum_{j=0}^{i-1}\big(\mathsf{IfThenElse}(j \bmod 2 = 0, j, 0)\big)}
\end{aligned}$$

  Para que $i+2$ sea par, tiene que ocurrir que i también lo sea. Como i es par, $i+1$ es impar por lo que se puede sacar de la sumatoria sin afectar el resultado.

- $I \land B \to wp(\textbf{ciclo}, I)$.
  - $I \land B \Rightarrow 0 \leq i \leq n+1 \land i < n$. Si $0 \leq i$ entonces $0 \leq i+2$. Por el otro lado, $i < n$ que es lo mismo que decir $i+2 < n+2 \equiv i+2 \leq n+1$. Juntando todo llegamos a $0 \leq i+2 \leq n+1$ como dice la WP ✓
  - $i \bmod 2 = 0$. está idéntico en I y en la WP ✓

- $res = \sum_{j=0}^{i-1} \big(\mathsf{IfThenElse}(j \bmod 2 = 0, j, 0)\big)$ está idéntico en I y en la WP ✓

3. $(I \wedge \neg B) \rightarrow Q_c$. Hay que demostrar que vale Q_c sabiendo que vale $(I \wedge \neg B)$. Donde $\neg B \equiv i \geq n$.
   - $(n \bmod 2 = 0 \rightarrow i = n) \wedge (n \bmod 2 = 1 \rightarrow i = n + 1)$. Analicemos los dos casos por separado.
     - Par. $n \bmod 2 = 0$. Quiero ver que $i = n$.
       - Sabemos por I que $i \leq n + 1$ y que $i \bmod 2 = 0$
       - Además, por $\neg B$, sabemos que $i \geq n$, por lo que $i = n \vee i = n + 1$
       - Pero como i y n son pares, tenemos que $i = n$ ✓
     - Impar. $n \bmod 2 = 1$. Quiero ver que $i = n + 1$.
       - Sabemos por I que $i \leq n + 1$ y que $i \bmod 2 = 0$
       - Además por $\neg B$ sabemos que $i \geq n$, por lo que $i = n \vee i = n + 1$
       - Pero como i es par y n es impar, tenemos que $i = n + 1$ ✓
   - $i \bmod 2 = 0$. Trivialmente cierto por I ✓
   - $res = \sum_{j=0}^{n-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))$.
     - Por I sabemos que $res = \sum_{j=0}^{i-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))$
     - Además sabemos que $(n \bmod 2 = 0 \rightarrow i = n) \wedge (n \bmod 2 = 1 \rightarrow i = n + 1)$
     - Veamos que ocurre en ambos casos donde quiero ver que $res = \sum_{j=0}^{n-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))$
       - Par. $n \bmod 2 = 0$
         - Sabemos que $i = n$
         - Por I tenemos que $res = \sum_{j=0}^{i-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0)) \Rightarrow res = \sum_{j=0}^{n-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))$
       - Impar. $n \bmod 2 = 1$.
         - Sabemos que $i = n + 1$
         - Por I tenemos que $res = \sum_{j=0}^{i-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))$
         - Reemplazando nos queda que

$$
\begin{aligned}
res &= \sum_{j=0}^{n}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0)) = \\
&= \sum_{j=0}^{n-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0)) + (\mathsf{IfThenElse}(n \mod 2 = 0, n, 0)) = \\
&= \sum_{j=0}^{n-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0)) + 0 = \\
&= \boxed{\sum_{j=0}^{n-1}(\mathsf{IfThenElse}(j \mod 2 = 0, j, 0))}
\end{aligned}
$$

         - Al extraer el último elemento de la sumatoria, como n es impar, el resultado del condicional es 0.

c) Función variante propuesta: $f_v = n + 1 - i$

4. $\{I \wedge B \wedge v_0 = f_v\}$ **ciclo** $\{f_v < v_0\}$. Queremos nuevamente ver que la tripla de Hoare planteada es válida. Es decir, que $(I \wedge B \wedge v_0 = f_v) \rightarrow wp(ciclo, f_v < v_0)$.

$$
\begin{aligned}
wp(ciclo, f_v < v_0) &\equiv wp(S1; S2, f_v < v_0) \\
&\equiv wp(S1, wp(S2, n + 1 - i < v_0)) \\
&\equiv wp(S1, (n - i - 1 < v_0)) \\
&\equiv n - i - 1 < v_0
\end{aligned}
$$

   - $(I \wedge B \wedge v_0 = f_v) \rightarrow wp(ciclo, f_v < v_0)$. Vemos que por $v_0 = f_v$ tenemos que $n - i - 1 < n - i \leftrightarrow -1 < 0$ que es trivialmente cierto.

5. $I \wedge f_v \leq 0 \rightarrow \neg B$. Donde $\neg B \equiv i \geq n$. De la función variante que $n + 1 - i \leq 0 \leftrightarrow n + 1 \leq i$. Si $n + 1 \leq i \leq n + 1$ entonces tenemos que $i = n + 1$. Se puede ver que $i = n + 1 \rightarrow i \geq n$.

Entonces el ciclo termina y, por consiguiente, es correcto.

Ejercicio 11. Considere el problema `sumaDivisores`, dado por la siguiente especificación:

```
proc sumaDivisores(in n : ℤ) : ℤ {
    requiere { n ≥ 1 }
    asegura { res = Σ_{j=1}^{n} (IfThenElse(n  mód j = 0, j, 0)) }
}
```

a) Escribir un programa en SmallLang que satisfaga la especificación del problema y que contenga exactamente un ciclo.

b) Escribir la pre y post condición del ciclo y su invariante.

c) Considere el siguiente invariante para este problema

$$I \equiv 1 \leq i \leq n/2 \wedge res = \sum_{j=1}^{i} (\mathsf{IfThenElse}(n \mod j = 0, j, 0))$$

Si no coincide con el propuesto en el inciso anterior, ¿qué cambios se le deben hacer al programa para que lo represente este invariante? ¿Deben cambiar la pre y post condición?

Solución

a)
```
i := 1;
res := 0;
while (i < n + 1) do
  if (n mod i = 0) then
    res := res + i;
  else
    skip;
  endif;
  i := i + 1;
endwhile;
```

b) $P_c \equiv \{n \geq 1 \wedge res = 0 \wedge i = 1\}$
$Q_c \equiv res = \sum_{j=1}^{n} (\mathsf{IfThenElse}(n \mod j = 0, j, 0))$
$I \equiv 1 \leq i \leq n+1 \wedge res = \sum_{j=1}^{i-1} (\mathsf{IfThenElse}(n \mod j = 0, j, 0)))$

c) Se puede cambiar la guarda del ciclo reemplazando `i < n + 1` por `i < n/2`. No hay que cambiar la pre y post condiciones dado que los divisores de un número entero están todos entre 1 y $n/2$

Ejercicio 12. Considere la siguiente especificación e implementación del problema `copiarSecuencia`, y la pre y post condiciones del ciclo.

Especificación

```
proc copiarSecuencia(in s : array⟨ℤ⟩, inout r : array⟨ℤ⟩) {
    requiere { |s| = |r| ∧ r = R_0 }
    asegura {
        |s| = |r| ∧_L (∀j : ℤ) (0 ≤ j < |s| →_L s[j] = r[j])
    }
}
```

Implementación en SmallLang

```
i := 0;
while (i < s.size()) do
  r[i]:=s[i];
  i:=i+1
endwhile
```

$$P_c \equiv |s| = |r| \wedge i = 0$$

$$Q_c \equiv |s| = |r| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < |r| \rightarrow_L s[j] = r[j])$$

a) ¿Qué variables del programa deben aparecer en el invariante?

b) Proponer un invariante e indicar qué cláusula del mismo es necesario para cada paso de la demostración.

c) Proponer una función variante y demostrar que el ciclo termina.

Solución

a) Las variables que deben aparecer son i , r y s

b)
- $I \equiv \{0 \leq i \leq |s| \land |s| = |r| \land (\forall j : \mathbb{Z})\ (0 \leq j < i \rightarrow_L s[j] = r[j]))\}$
- Para poder demostrar $P_c \rightarrow I$ solo necesito $0 \leq i \leq |s|$
- Para poder demostrar $\{I \land B\}S\{I\}$ solo necesito $0 \leq i \leq |s|$
- Ahora para demostrar $(I \land \neg B) \rightarrow Q_c$ se necesita el invariante completo

c) Defino $f_v = |s| - i$ Para probar finalización del ciclo tengo que probar:

- $(I \land f_v \leq 0) \rightarrow \neg B$
  $(I \land f_v \leq 0) \rightarrow |s| - i \leq 0 \rightarrow i \geq |s| \equiv \neg B$
- $\{I \land B \land f_v = v_0\}S\{f_v < v_0\}$
  $\{I \land B \land f_v = v_0\}S\{f_v < v_0\} \iff (I \land B \land f_v = v_0) \rightarrow wp(S, f_v < v_0)$

$$\begin{aligned} wp(S, I) &\equiv wp(r[i] = s[i], wp(i = i + 1, |s| - i < v_0)) \\ &\equiv wp(r[i] = s[i], |s| - (i + 1) < v_0) \\ &\equiv |s| - (i + 1) < v_0 \end{aligned}$$

Pero $f_v = v_0 \rightarrow v_0 - 1 < v_0 \iff -1 < 0$

Ejercicio 13. Sea el siguiente ciclo con su correspondiente precondición y postcondición:

```
while (i >= s.size() / 2) do
  suma := suma + s[s.size()-1-i];
  i := i - 1
endwhile
```

$$P_c : \{|s| \mod 2 = 0 \land i = |s| - 1 \land suma = 0\}$$

$$Q_c : \{|s| \mod 2 = 0 \land i = |s|/2 - 1 \ \land_L \ suma = \sum_{j=0}^{|s|/2-1} s[j]\}$$

a) Proponer un invariante e indicar qué clausula del mismo es necesaria para cada paso de la demostración.

b) Proponer una función variante que permita demostrar que el ciclo termina.

c) Demostrar la terminación del ciclo utilizando la función variante.

Solución

a) $I \equiv \{|s|/2 - 1 \leq i \leq |s| - 1 \land |s| \mod 2 = 0 \land suma = \sum_{j=0}^{|s|-i-2} s[j]\}$

- Para poder demostrar $P_c \rightarrow I$ solo necesito $|s|/2 - 1 \leq i \leq |s| - 1$
- Para poder demostrar $\{I \land B\}S\{I\}$ solo necesito $|s|/2 - 1 \leq i \leq |s| - 1$
- Ahora para demostrar $(I \land \neg B) \rightarrow Q_c$ se necesita el invariante completo

b) $f_v = i - (|s|/2 - 1)$ Para probar finalización del ciclo tengo que probar:

- $(I \land f_v \leq 0) \rightarrow \neg B$
  $|s|/2 - 1 \leq i \leq |s|-1 \land i - (|s|/2 - 1) \leq 0 \leftrightarrow$
  $|s|/2 - 1 \leq i \land i \leq |s|/2 - 1 \leftrightarrow$
  $i = |s|/2 - 1 \rightarrow i < |s|/2 \equiv \neg B$

- $\{I \wedge B \wedge f_v = v_0\} S \{f_v < v_0\}$
  $\leftrightarrow ((|s|/2 - 1 \leq i \leq |s| - 1 \wedge |s| \text{ mód } 2 = 0 \wedge suma = \sum_{j=0}^{|s|-i-2} s[j]) \wedge i \geq |s|/2 \wedge i - (|s|/2 - 1) = v_0) \rightarrow$
  $wp(S, i - (|s|/2 - 1) < v_0)$

  Donde $wp(S, i - |s|/2 - 1 < v_0) \equiv 0 \leq i < |s| \wedge i - 1 - (|s|/2 - 1) < v_0$

  Luego, podemos ver que:

  $(|s|/2 - 1 \leq i \leq |s| - 1) \rightarrow (0 \leq i < |s|)$
  $(i - (|s|/2 - 1) = v_0) \rightarrow (i - 1 - (|s|/2 - 1) < v_0)$

Ejercicio 14. Dado el siguiente problema

```
proc sumarElementos(in s : array⟨ℤ⟩) : ℤ {
   requiere { |s| ≥ 1 ∧ |s|  mód 2 = 0 }
   asegura { res = Σ_{j=0}^{|s|-1} s[j] }
}
```

Dar un invariante y función variante para cada una de estas posibles implementaciones

a)
```
res := 0
i := 0
while (i < s.size()) do
  res := res + s[i];
  i := i + 1
endwhile
```

b)
```
res := 0
i := 0
while (i < s.size()) do
  res := res + s[s.size() - 1 - i];
  i := i + 1
endwhile
```

c)
```
res := 0
i := s.size() - 1
while (i >= 0) do
  res := res + s[i];
  i := i - 1
endwhile
```

d)
```
res := 0
i := 0
while (i < s.size() / 2) do
  res := res + s[i] + s[s.size() - 1 - i];
  i := i + 1
endwhile
```

Solución

a)
- $I \equiv \{0 \leq i \leq |s| \wedge_L res = \sum_{j=0}^{i-1} s[j]\}$
- $f_v = |s| - i$

b)
- $I \equiv \{0 \leq i \leq |s| \wedge_L res = \sum_{j=0}^{i-1} s[|s| - 1 - j]\}$
- $f_v = |s| - i$

c)
- $I \equiv \{-1 \leq i \leq |s| - 1 \wedge_L res = \sum_{j=i+1}^{|s|-1} s[j]\}$
- $f_v = i + 1$

d)
- $I \equiv \{0 \leq i \leq |s|/2 \wedge_L res = \sum_{j=0}^{i-1} (s[j] + s[|s| - 1 - j])\}$
- $f_v = |s|/2 - i$

Ejercicio 15. Considerando el siguiente Invariante:

$$I \equiv \{0 \leq i \leq |s| \wedge (\forall j : \mathbb{Z})\ (0 \leq j < i \rightarrow_L (j \mod 2 = 0 \wedge s[j] = 2 \times j \vee j \mod 2 \neq 0 \wedge s[j] = 2 \times j + 1))\}$$

- Escribir un programa en SmallLang que se corresponda al invariante dado.
- Defina las P_c, B y Q_c que correspondan a su programa.
- Dar una función variante para que se pueda completar la demostración.

Solución

a)
```
i := 0;
while (i < s.size()) do
  if (i mod 2 = 0) then
    s[i] := 2*i;
  else
    s[i] := 2*i +1;
  endif;
  i := i + 1;
endwhile;
```

b) $P_c \equiv i = 0$
$B \equiv i < |s|$
$Q_c \equiv (\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L ((j \mod 2 = 0 \land s[j] = 2 \times j) \lor (j \mod 2 \neq 0 \land s[j] = 2 \times j + 1)))$

c) $f_v = |s| - i$

Ejercicio 16. Considerando el siguiente Invariante:

$$I \equiv \{0 \leq i \leq |s|/2 \land (\forall j : \mathbb{Z})\ (0 \leq j < i) \rightarrow_L (s[j] = 0 \land s[|s| - j - 1] = 0)\}$$

- Escribir un programa en SmallLang que se corresponda al invariante dado.
- Defina las P_c, B y Q_c que correspondan a su programa.
- Dar una función variante para que se pueda completar la demostración.

Solución

a)
```
i := 0;
while (i < s.size()/2) do
    s[i] := 0;
    s[s.size()-1-j] := 0;
  i := i + 1;
endwhile;
```

b)
- $P_c \equiv \{i = 0 \land |s| \mod 2 = 0\}$
- $B \equiv \{i < |s|/2\}$
- $Q_c \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < |s| \rightarrow_L s[j] = 0)\}$

c) $f_v = |s|/2 - i$

Ejercicio 17. Indique si el siguiente enunciado es verdadero o falso; fundamente:
Si dados B y I para un ciclo S existe una función f_v que cumple lo siguiente:

- $\{I \land B \land f_v = V_0\} S \{f_v > V_0\}$
- $(\exists k : \mathbb{Z})\ (I \land f_v \geq k \rightarrow \neg B)$

entonces el ciclo siempre termina.

Solución

Verdadero, si tomamos a $f'_v = k - f_v$ podemos ver que esta nueva función variante cumple el teorema de Terminacion

Ejercicio 18. Considere la especificación de la función `existeElemento` y su implementación

Especificación

```
proc existeElemento(in s : array⟨ℤ⟩, in e : ℤ) : bool {
    requiere { True }
    asegura {
        res = True ↔
        (∃k : ℤ) (0 ≤ k < |s|) ∧_L s[k] = e)
    }
}
```

Implementación en SmallLang

```
i := 0;
j := −1;
while (i < s.size()) do
  if (s[i] = e) then
    j := i
 else
   skip
 endif;
 i := i + 1
endwhile;
if (j != −1)
  res := true
else
  res := false
endif
```

Escribir los pasos necesarios para demostrar la correctitud de la implementación respecto a la especificación usando WP y el teorema del invariante

Solución

a) $P \to wp(S, Q)$

b) Teorema del Invariante
$P_c \equiv i = 0 \land j = -1$
$Q_c \equiv res = True \leftrightarrow ((\exists k : \mathbb{Z})\ (0 \le k < |s|) \land_L s[k] = e)$
$I \equiv 0 \le i \le |s| \land (j \neq -1 \to (\exists k : \mathbb{Z})\ (0 \le k < i \land_L s[k] = e)) \land (j = -1 \to (\exists k : \mathbb{Z})\ (0 \le k < i \land_L s[k] \neq e))$
$B \equiv i \ge |s|$

- $P_c \to I$
  Se que $i = 0 \land j = -1$ luego $0 \le 0 \le |s|$ es verdadero, la implicación de $j \neq -1$ es verdadera, la implicación de $j = -1$ es verdadera pues el antecedente del $\exists$ es falso.
- $(I \land \neg B) \to Q_c$
  $(I \land \neg B) \to i = |s|$ y por lo tanto para este valor de i, las dos ramas del invariante son iguales a las de Q_c
- $\{I \land B\}S\{I\}$
  $\{I \land B\}S\{I\} \leftrightarrow (I \land B) \to wp(S, I)$

c) $Q_c \to wp(C, Post)$

$$
\begin{aligned}
wp(if(...), Post) &\equiv def(j \neq -1) \land_L (j \neq -1 \land wp(r = True, Post)) \lor \\
&\qquad (j = -1 \land wp(r = false, Post)) \\
&\equiv (j \neq -1 \land wp(r = true, Post)) \lor \\
&\quad (j = -1 \land wp(r = false, Post)) \\
&\equiv (j \neq -1 \land true = true \leftrightarrow (\exists k : \mathbb{Z})\ (0 \le k < |s| \land_L s[k] = e)) \lor \\
&\quad (j = -1 \land false = true \leftrightarrow (\exists k : \mathbb{Z})\ (0 \le k < |s| \land_L s[k] \neq e))
\end{aligned}
$$

4.2.2. Ejercicios de parcial

Ejercicio 19. Dados los siguientes ciclos y sus respectivas precondición (P_c) y poscondición (Q_c).

1. Proponer un invariante (I) y una función variante (f_v) para el ciclo
2. Demostrar los siguientes pasos de la demostración de correctitud del ciclo
   i) $P_c \to I$
   ii) $(I \wedge \neg B) \to Q_c$
   iii) $(I \wedge f_v \leq 0) \to \neg B$

a) $P_c \equiv \{s = S_0 \wedge i = 0 \wedge 0 \leq d < |s|\}$

```
while ( i < d )
  s[ i ] := e
  i := i+1
endwhile
```

$Q_c \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < d \to_L s[j] = e) \wedge (\forall j : \mathbb{Z})\ (d \leq j < |s| \to_L s[j] = S_0[j]))\}$

b) $P_c \equiv \{s = S_0 \wedge 0 \leq d < |s| \wedge i = d\}$

```
while ( i < |s| )
 s[ i ] := e
 i := i+1
endwhile
```

$Q_c \equiv \{(\forall j : \mathbb{Z})\ (0 \leq j < d \to_L s[j] = S_0[j]) \wedge (\forall j : \mathbb{Z})\ (d \leq j < |s| \to_L s[j] = e)\}$

c) $P_c \equiv \{i = |s| - 1 \wedge res = 0\}$

```
while (i ≥ 0)
  res := res + s[ i ] + 1
  i := i - 1
endwhile
```

$Q_c \equiv \{res = |s| + \sum_{j=0}^{|s|-1} s[j]\}$

Solución

a) 1.
- $I \equiv \{0 \leq i \leq d \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < i) \to_L (s[j] = e) \wedge (\forall j : \mathbb{Z})\ (i \leq j < |s| \to_L s[j] = S_0[j]))\}$
- $f_v = d - i$

2. i) $P_c \to I$:
   - $i = 0 \to 0 \leq 0 \leq d$ ✓
   - $i = 0 \to 0 \leq j < 0 \equiv False \Rightarrow (\forall j : \mathbb{Z})\ (\overbrace{0 \leq j < 0}^{False}) \to_L (s[j] = e) \equiv True$ ✓
   - $i = 0 \wedge s = S_0 \to (\forall j : \mathbb{Z})\ (0 \leq j < |s|) \to_L (s[j] = S_0[j]) \equiv True$ ✓

   ii) $(I \wedge \neg B) \Rightarrow Q_c$:
   - $(I \wedge \neg B) \Rightarrow (0 \leq i \leq d \wedge i \geq d) \equiv i = d$
   - $i = d \wedge (\forall j : \mathbb{Z})\ (0 \leq j < i) \to_L (s[j] = e) \Rightarrow (\forall j : \mathbb{Z})\ (0 \leq j < d) \to_L (s[j] = e)$ ✓
   - $i = d \wedge (\forall j : \mathbb{Z})\ (i \leq j < |s|) \to_L (s[j] = S_0[j]) \Rightarrow (\forall j : \mathbb{Z})\ (d \leq j < |s|) \to_L (s[j] = S_0[j])$ ✓

   iii) $(I \wedge f_v \leq 0) \to \neg B$
   - $(I \wedge f_v \leq 0) \Rightarrow (0 \leq i \leq d \wedge d - i \leq 0) \equiv (0 \leq i \leq d \wedge d \leq i) \Rightarrow i = d \Rightarrow \neg(i < d) \equiv \neg B$

b) 1.
- $I \equiv \{d \leq i \leq |s| \wedge_L (\forall j : \mathbb{Z})\ (0 \leq j < d) \to_L (s[j] = S_0[j]) \wedge (\forall j : \mathbb{Z})\ (d \leq j < i) \to_L (s[j] = e))\}$
- $f_v = |s| - i$

2. i) $P_c \to I$:
   - $i = d \to d \leq d \leq |s|$ ✓
   - $i = d \wedge s = S_0 \Rightarrow (\forall j : \mathbb{Z})\ (0 \leq j < d) \to_L (s[j] = S_0[j]) \equiv True$ ✓
   - $i = d \to d \leq j < d \equiv False \to (\forall j : \mathbb{Z})\ (\overbrace{d \leq j < d}^{False}) \to_L (s[j] = S_0[j]) \equiv True$ ✓

ii) $(I \wedge \neg B) \Rightarrow Q_c$:
- $(I \wedge \neg B) \Rightarrow (d \leq i \leq |s| \wedge i \geq |s|) \equiv i = |s|$
- $i = |s| \wedge (\forall j : \mathbb{Z})\ (d \leq j < i) \rightarrow_L (s[j] = e) \Rightarrow (\forall j : \mathbb{Z})\ (d \leq j < |s|) \rightarrow_L (s[j] = e)$ ✓
- $(\forall j : \mathbb{Z})\ (0 \leq j < d) \rightarrow_L (s[j] = S_0[j])$ es una cláusula del Invariante ✓

iii) $(I \wedge f_v \leq 0) \rightarrow \neg B$
- $(I \wedge f_v \leq 0) \Rightarrow (d \leq i \leq |s| \wedge |s| - i \leq 0) \equiv (d \leq i \leq |s| \wedge |s| \leq i) \Rightarrow i = |s| \Rightarrow \neg(i < |s|) \equiv \neg B$

c) 1.
- $I \equiv \{-1 \leq i \leq |s| - 1 \wedge_L res = \sum_{j=i+1}^{|s|-1} (s[j] + 1)\}$
- $f_v = i + 1$

2. i) $P_c \rightarrow I$:
- $i = |s| - 1 \rightarrow res = \sum_{j=|s|}^{|s|-1} (s[j] + 1) = 0$ ✓

ii) $(I \wedge \neg B) \Rightarrow Q_c$:
- $(I \wedge \neg B) \Rightarrow (-1 \leq i \leq |s| - 1 \wedge i < 0) \equiv i = -1 \Rightarrow$
  $res = \sum_{j=0}^{|s|-1} (s[j] + 1) = \sum_{j=0}^{|s|-1} s[j] + \sum_{j=0}^{|s|-1} 1 = \sum_{j=0}^{|s|-1} s[j] + |s|$ ✓

iii) $(I \wedge f_v \leq 0) \rightarrow \neg B$
- $(I \wedge f_v \leq 0) \Rightarrow (-1 \leq i \leq |s| - 1 \wedge i + 1 \leq 0) \equiv (-1 \leq i \leq |s| - 1 \wedge i \leq -1) \Rightarrow i = -1 \Rightarrow \neg(i \geq 0) \equiv \neg B$

Ejercicio 20. Dado el siguiente programa con su especificación

$P_c \equiv \{n > 0 \wedge res = 1 \wedge i = 1\}$

```
while( i < n)
    res := res * i
    i := i+1;
endwhile
```

$Q_c \equiv \{res = (n-1)!\}$

a) Escriba el Invariante del ciclo

b) Demuestre formalmente que el invariante propuesto cumple con las hipótesis del Teorema del Invariante

c) Decida si las siguiente funciones pueden o no usarse como funciones variantes para demostrar la terminación del ciclo y justifique muy brevemente por qué.

(i) $f_v = n$

(ii) $f_v = n - i$

(iii) $f_v = n - 1 - i$

(iv) $f_v = i - n$

(v) $f_v = (n-1)! - res$

Solución

a) $I \equiv \{1 \leq i \leq n \wedge res = (i-1)!\}$

b) Demostración formal de las hipótesis del Teorema del Invariante:

i) $P_c \rightarrow I$:
- $n > 0 \wedge i = 1 \rightarrow 1 \leq 1 \leq n \equiv True$ ✓
- $res = 1 \wedge i = 1 \rightarrow 1 = (1-1)! \equiv 1 = 0! \equiv 1 = 1 \equiv True$ ✓

ii) $\{I \wedge B\}S\{I\} \leftrightarrow I \wedge B \rightarrow wp(S, I)$:
- $wp(S, I) \equiv wp(\textbf{res := res * i},\ wp(\textbf{i := i + 1},\ 1 \leq i \leq n \wedge res = (i-1)!))$
- $wp(S, I) \equiv wp(\textbf{res := res * i},\ 1 \leq i + 1 \leq n \wedge res = i!)$
- $wp(S, I) \equiv 1 \leq i + 1 \leq n \wedge res \times i = i!$
- $I \wedge B \rightarrow 1 \leq i \leq n \wedge i < n \rightarrow 1 \leq i \leq n - 1 \rightarrow 2 \leq i + 1 \leq n \rightarrow 1 \leq i + 1 \leq n$ ✓

- $I \wedge B \rightarrow res = (i-1)! \rightarrow res \times i = (i-1)! \times i \equiv i!$ ✓

iii) $(I \wedge \neg B) \rightarrow Q_c$:

- $(I \wedge \neg B) \rightarrow 1 \leq i \leq n \wedge i \geq n \rightarrow i = n$ ✓
- $i = n \wedge res = (i-1)! \rightarrow res = (n-1)! \equiv Q_c$ ✓

c) Evaluación de las funciones variantes propuestas:

(i) $f_v = n$: **NO.** Es una constante, su valor jamás decrece con las iteraciones.

(ii) $f_v = n - i$: **SÍ.** Decrece en 1 por cada iteración. Además, $(I \wedge f_v \leq 0) \rightarrow n - i \leq 0 \rightarrow i \geq n \rightarrow \neg B$.

(iii) $f_v = n - 1 - i$: **NO.** Si bien decrece, podría llegar a ser 0 sin que la guarda sea falsa. Si $i = n - 1$, $f_v = 0$, pero $i < n$ es $True$. Esto rompe la demostración de terminación.

(iv) $f_v = i - n$: **NO.** Su valor aumenta en cada iteración en lugar de disminuir.

(v) $f_v = (n-1)! - res$: Es decreciente en cada iteración. Además, $(I \wedge f_v \leq 0) \equiv (n-1)! - res \leq 0 \wedge 1 \leq i \leq n \wedge res = (i-1)! \equiv (n-1)! \leq (i-1)! \wedge 1 \leq i \leq n \wedge res = (i-1)!$. Y tenemos que: $(n-1)! \leq (i-1)! \rightarrow i \geq n \equiv \neg B$.

Ejercicio 21. Dado el siguiente programa con su especificación

$P_c \equiv \{n > 0 \wedge res = 1 \wedge i = 1 \wedge j = 2\}$

```
while( i < n)
  res := res + j
  j := j*2;
  i := i+1;
endwhile
```

$Q_c \equiv \{res = \sum_{k=0}^{n-1} 2^k\}$

Contamos con el siguiente invariante, que sabemos que es incorrecto:
$I \equiv \{0 \leq i \leq n \wedge j = 2^{i+1} \wedge res = \sum_{k=0}^{i} 2^k\}$

a) Señale cuáles hipótesis del teorema del invariante no se cumplen, sin usar demostraciones formales.

b) Escriba un invariante que resulte correcto y demuestre formalmente **únicamente** los axiomas que no se cumplían con el invariante anterior.

Solución

a) Las hipótesis del Teorema del Invariante que no se cumplen son:

i) $P_c \rightarrow I$: Al inicializar el ciclo tenemos $i = 1$, $j = 2$ y $res = 1$. Sin embargo, el invariante erróneo exige que $j = 2^{1+1} = 4$ (falla) y que $res = \sum_{k=0}^{1} 2^k = 1 + 2 = 3$ (falla, porque res arranca en 1).

ii) $(I \wedge \neg B) \rightarrow Q_c$: Al salir del ciclo sabemos que $\neg B \equiv i \geq n$. Si asumimos que $i = n$, el invariante erróneo nos asegura que $res = \sum_{k=0}^{n} 2^k$. Pero Q_c requiere que el límite superior de la sumatoria sea $n - 1$. Al sobrar un término en la sumatoria, la implicación falla.

b) Invariante correcto propuesto:
$I \equiv \{1 \leq i \leq n \wedge j = 2^i \wedge res = \sum_{k=0}^{i-1} 2^k\}$

Demostración formal **únicamente** de las hipótesis que no se cumplían con el invariante anterior:

i) $P_c \rightarrow I$:

- $n > 0 \wedge i = 1 \rightarrow 1 \leq 1 \leq n \equiv True$ ✓
- $j = 2 \wedge i = 1 \rightarrow 2 = 2^1 \equiv True$ ✓
- $res = 1 \wedge i = 1 \rightarrow 1 = \sum_{k=0}^{1-1} 2^k \rightarrow 1 = \sum_{k=0}^{0} 2^k \rightarrow 1 = 2^0 \rightarrow 1 = 1 \equiv True$ ✓

ii) $(I \wedge \neg B) \rightarrow Q_c$:

- $(I \wedge \neg B) \rightarrow 1 \leq i \leq n \wedge i \geq n \rightarrow i = n$ ✓
- $i = n \wedge res = \sum_{k=0}^{i-1} 2^k \rightarrow res = \sum_{k=0}^{n-1} 2^k \equiv Q_c$ ✓

Ejercicio 22. Dados

- La función $fibonacci : \mathbb{N} \to \mathbb{N}$ definida como:

$$fibonacci(n) = \begin{cases} 1 & \text{si } n = 0 \text{ o } n = 1 \\ fibonacci(n-1) + fibonacci(n-2) & \text{en otro caso} \end{cases}$$

- El siguiente programa a completar, con su especificación:

$P \equiv \{n > 0\}$

```
proc fib(int n)
  s := new array<int>(n+1)
  s[0] := 1
  s[1] := 1

  i := *A COMPLETAR*
  while(*A COMPLETAR*)
    *A COMPLETAR*
  endwhile

  return s[i-1]
end proc
```

$Q_c \equiv \{res = fibonacci(n)\}$

- El siguiente invariante correcto para el ciclo dentro del programa:

$I \equiv \{n > 0 \land |s| = n + 1 \land_L s[0] = 1 \land s[1] = 1 \land$
$2 \leq i \leq |s| \land (\forall j : \mathbb{Z})\ (2 \leq j < i \to_L s[j] = s[j-1] + s[j-2])\}$

Se pide:

a) Completar el código en base al invariante y la especificación del programa

b) Explicar, sin demostrar formalmente, por qué este invariante cumple con los tres puntos del teorema del invariante

Solución

a) **a) Código completado:**

```
proc fib(int n)
    s := new array<int>(n+1)
    s[0] := 1
    s[1] := 1

    i := 2
    while(i <= n) do
        s[i] := s[i-1] + s[i-2]
        i := i + 1
    endwhile

    return s[i-1]
end proc
```

b) **b) Por qué cumple el Teorema del Invariante:**

i) **Inicialización ($P_c \to I$):** Al comenzar, $i = 2$. El rango de cotas $2 \leq 2 \leq |s|$ se cumple (pues $|s| = n + 1$). El cuantificador universal (`paraTodo`) tiene un rango vacío ($2 \leq j < 2$), por lo que es trivialmente verdadero.

ii) **Preservación ($I \land B \to wp$):** En cada iteración, se asigna a `s[i]` exactamente la suma de los dos valores anteriores, lo que satisface la propiedad para $j = i$. Al incrementar i, se expande la "ventana" de la sumatoria validada.

iii) **Terminación y Q_c:** Al salir del ciclo, $\neg B \equiv i > n$. Como por invariante $i \leq n + 1$, se concluye que $i = n + 1$. Al retornar `s[i-1]`, estamos devolviendo `s[n]`, que por la sucesión armada sabemos que es exactamente `fibonacci(n)`.

Ejercicio 23. Dados la siguiente precondición, postcondición e invariante de un ciclo

$P_c \equiv \{n > 0 \land i = 0 \land j = 2 \times n \land res = 0\}$

$I \equiv \{0 \leq i \leq n \land n \leq j \leq 2 \times n \land res = \sum_{s=0}^{i-1} s + \sum_{t=j+1}^{2n} t\}$

$Q_c \equiv \{res = \sum_{s=0}^{n-1} s + \sum_{t=n+1}^{2n} t\}$

Y el siguiente programa incompleto

```
proc DobleNCuadrado(int n)
  res := 0
  i := 0
  j := 2*n

  while [COMPLETAR]
      [COMPLETAR]
  endwhile

  return res
end proc
```

Se pide

a) Completar el programa para que corresponde al invariante dado

b) Escribir la precondición del programa

c) Explicar sin hacer las demostraciones formales, por qué este invariante cumple con los tres puntos del teorema del invariante

Solución

a) **a) Código completado:**

```
proc DobleNCuadrado(int n)
    res := 0
    i := 0
    j := 2*n

    while (i < n) do
        res := res + i + j
        i := i + 1
        j := j - 1
    endwhile

    return res
end proc
```

b) **b) Precondición:**
$P_c \equiv \{n \geq 0\}$

c) **c) Por qué cumple el Teorema del Invariante:**

i) **Inicialización:** Al arrancar, $i = 0$ y $j = 2n$. Los rangos de ambas sumatorias del invariante quedan invertidos/vacíos ($\sum_0^{-1}$ y $\sum_{2n+1}^{2n}$), por lo que $res = 0 + 0 = 0$, coincidiendo perfectamente con la asignación inicial.

ii) **Preservación:** En cada paso, el cuerpo suma i y j a `res`. Al incrementar i y decrementar j, obligamos a que ambas sumatorias agreguen exactamente un nuevo término en sus respectivos extremos de los rangos evaluados.

iii) **Terminación:** Al salir del ciclo, $i = n$. Como i avanza n pasos hacia adelante desde 0 y j retrocede n pasos hacia atrás desde $2n$, al final $j = n$. Reemplazando i y j por n en el invariante, los topes de las sumatorias quedan exactamente iguales a los descritos en Q_c.

Ejercicio 24. Dadas las siguientes funciones variantes con un n positivo fijo, escriba un ciclo lo más sencillo posible tal que estas funciones resulten correctas para demostrar la terminación del ciclo y explique por qué. De ser necesario defina también el valor inicial de todas las variables. Si la función no se puede usar para ningún ciclo explique por qué.

a) $f_v = n - 2 \times i$

b) $f_v = i - n$

c) $f_v = n - i + 1$

d) $f_v = n + i$

e) $f_v = n + (n - i) + i$

f) $f_v = n - i - j$

Solución

a) **a)** $f_v = n - 2 \times i$

```
i := 0
while (2 * i < n) do
    i := i + 1
endwhile
```

Justificación: Como i incrementa en 1, el factor $2 \times i$ incrementa en 2, forzando a que la función f_v decrezca estrictamente. Al ser $f_v \leq 0 \rightarrow 2i \geq n \rightarrow \neg B$, asegura terminación.

b) **b)** $f_v = i - n$

```
i := 2 * n
while (i > n) do
    i := i - 1
endwhile
```

Justificación: Inicia con i por encima de n. Al decrementar i, f_v decrece estrictamente hacia 0. Si $f_v \leq 0 \rightarrow i \leq n \rightarrow \neg B$.

c) **c)** $f_v = n - i + 1$

```
i := 0
while (i <= n) do
    i := i + 1
endwhile
```

Justificación: Es el variante ideal para ciclos donde la guarda incluye la igualdad ($i \leq n$). Como i crece, f_v decrece. Si $f_v \leq 0 \rightarrow i \geq n + 1 \rightarrow i > n \rightarrow \neg B$.

d) **d)** $f_v = n + i$

```
i := 0
while (i > -n) do
    i := i - 1
endwhile
```

Justificación: Para que este variante decrezca en un marco positivo, i debe ir disminuyendo hacia un tope negativo. $f_v \leq 0 \rightarrow i \leq -n \rightarrow \neg B$.

e) **e)** $f_v = n + (n - i) + i$

No se puede usar. Operando algebraicamente se simplifica a $f_v = 2n$. Al ser una constante respecto a las variables de iteración, nunca va a decrecer estrictamente, requisito fundamental para acotar un ciclo.

f) **f)** $f_v = n - i - j$

```
i := 0
j := 0
while (i + j < n) do
    i := i + 1
endwhile
```

Justificación: Decrece invariablemente porque i incrementa (también valdría si j incrementara o lo hicieran alternadamente). Si $f_v \leq 0 \rightarrow i + j \geq n \rightarrow \neg B$.